L A K B A Y

LAKBAY SALITA - OCTOBER 2026

How Lakbay Salita was built

The tools, the timeline, the technical ideas, the decisions and the problems solved

Contents

1. The project at a glance . . . 2

1.1 Numbers . . . 2

1.2 How to read this document . . . 2

2. Tools and way of working . . . 3

2.1 The working loop . . . 3

2.2 What the screenshots and builders gave the project . . . 3

3. Timeline from the Git history . . . 3

4. The build in eight phases . . . 5

Phase 1. Foundation and menu (March to April 2026) . . . 5

Phase 2. Dictionary and Library (August to September 2026) . . . 5

Phase 3. Wordle . . . 5

Phase 4. Crossword . . . 6

Phase 5. Word Search . . . 6

Phase 6. One data model for all games . . . 6

Phase 7. The learning layer (October 2026) . . . 6

Phase 8. Polish and delivery . . . 7

5. Technical ideas that hold the project together . . . 8

6. Decisions and why . . . 9

7. Problems met and how they were solved . . . 10

8. How the work was tested . . . 10

9. State of the project . . . 11

9.1 Code by module . . . 11

9.2 Known issues . . . 11

10. Lessons and next steps . . . 12

10.1 What worked . . . 12

10.2 What to do differently . . . 12

10.3 Next steps . . . 12

Appendix. Editor tools (Tools menu) . . . 13

1. The project at a glance

Lakbay Salita is a 2D word-learning game for Cebuano, Ilonggo and Tagalog, built in Unity 2021.3.45f2 for Windows. This document explains **how it was built**: the tools, the order of the work, the technical ideas that hold it together, the decisions that were made and the problems that had to be solved. It is based on a full read of the project and its Git history.

> **Short version.** The game grew from a menu prototype (March 2026) into three puzzle games, a dictionary, learning cards and a phrasebook (October 2026). The big technical turn was moving from one hand-made scene or script per level to **data (JSON) plus one play scene per game**, and rebuilding every screen with **editor builder scripts** so a design can be changed in one menu click.

1.1 Numbers

Item	Value
Engine and version	Unity 2021.3.45f2, built-in render pipeline, uGUI and TextMeshPro
Code	97 C# scripts, 20,572 lines (12 are editor tools)
Scenes	15 playable scenes (plus 128 archived older scenes kept for reference)
Content	63,540 dictionary entries; 126 Wordle levels; 63 crosswords; 63 word searches; 53 phrases
Art and audio	151 sprites, 11 prefabs, 7 music tracks, 25 sound effects
History	13 Git commits from 3 March 2026 to 15 October 2026, branches main and sub

1.2 How to read this document

- Sections 2 and 3: the tools and the timeline.
- Section 4: the work in eight phases, in the order it was done.
- Sections 5 to 7: the technical ideas, the decisions and the problems solved.
- Sections 8 and 9: how the work was tested and what state the project is in.
- Section 10: lessons and what to do next. For a recipe to rebuild the game, see the companion PDF "Step by step: how to create Lakbay Salita".

2. Tools and way of working

Tool	Used for
Unity 2021.3.45f2	The game engine and editor. Scenes, prefabs, UI (uGUI), TextMeshPro text, audio import, Windows build.
C# in Visual Studio Code	All scripts. Unity packages for the IDE (Rider, Visual Studio and VS Code support) are in the manifest.
Git and GitHub	History and backup. A new repository was created in October 2026 with two branches: main and sub.
Unity MCP package	Lets an AI assistant (Claude Code) drive the Unity editor: run editor code, open scenes, press Play, read the console and take screenshots.
Claude Code	Pair-programming assistant. It wrote and changed scripts, ran the builder tools, played the games in the editor and checked screens.
Python scripts	Generate audio (`BGM Ideas/make_bgm.py`, `SFX Ideas/make_sfx.py`), prepare and unify the JSON word lists, make mockup images (Pillow).
pptxgenjs, docx-js, reportlab	Presentation, Word and PDF documents about the project.
TextMeshPro	Roboto font family converted to SDF font assets.

2.1 The working loop

1. **Describe the change** (for example "make the Library slider light" or "add a Phrasebook").
2. **Show an idea as an image first** when the change is big. Mockups were drawn with Python and saved in the `Mockups` folder before any screen was built (learning ideas, Phrasebook, main menu styles).
3. **Build it** in code. Screens are written as builder scripts that create real scene objects, so the result can be seen in the editor before pressing Play.
4. **Test in Play mode.** A helper drove the editor: load a scene, click buttons, type letters, take a 1920 x 1080 screenshot, read the console.
5. **Commit to the sub branch first**, then ask whether to merge to main. This is a standing rule of the project.

2.2 What the screenshots and builders gave the project

Because every screen is created by a script, a colour or layout change is made once in the script and applied to every scene by running the builder. This is why the 30 Wordle level scenes and 30 Crossword level scenes could be redesigned together, and later replaced by a single play scene each.

3. Timeline from the Git history

Dates and sizes are taken from the 13 commits. The early commits are small; most of the code arrived in the last three.

Date	Commit message	What it contains
2026-03-03	main	First Unity project: TextMesh Pro essentials, .gitignore (116 files).
2026-03-11	menu / menu v1.0	First menu scenes and a red material (menu prototype).

Date	Commit message	What it contains
2026-04-13	menu (2 commits)	Small navigation scripts: library button, play to language scene, to main menu.
2026-08-26	Initial commit, 2 manual commits, merge	A new GitHub repository, the work brought together and merged.
2026-09-02	2026-09-02	260 files: Profile and Language Selection scenes, the first Wordle scene, word JSON files; a 275,000-line Tagalog dictionary file removed.
2026-09-17	09-17-26	738 files: Roboto fonts, ten-word lists for all three languages, more puzzle scenes.
2026-10-09	Restructure project, add category modes and Word Search	1,242 files: new Assets/Main folder layout, category modes (topics), JSON level files, Word Search, the full dictionary files.
2026-10-15	Add Learn cards, Phrasebook, button sounds, settings and Profile update	Learn cards, Phrasebook and Sentence builder, sound effects, gear settings, new Profile.
After 15 Oct	(not yet committed)	ScreenFit (window limits, 16:9 letterbox, maximize), project documents.

Note: commit messages from March to September are short, so the descriptions of those commits are read from the files they changed, not from the messages.

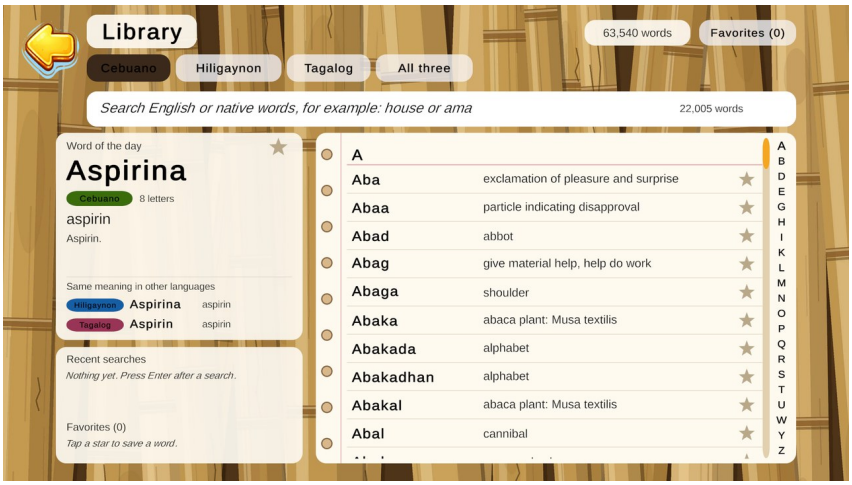
4. The build in eight phases

Phase 1. Foundation and menu (March to April 2026)

- Created the Unity project, imported TextMeshPro, and set up Git with a .gitignore for Library, Temp and Logs.
- Built the first menu scenes: Main menu, Profile, Language selection and Library, with small scripts that load scenes.
- Result: a clickable menu that moves between four screens. The idea of one scene per screen and a `SceneNavigator` script on a `SceneManager` object started here and is still used.

Phase 2. Dictionary and Library (August to September 2026)

- Collected word lists for the three languages as JSON files (`cebuano.json`, `hiligaynon.json`, `tagalog.json`). The first lists were small; by the October restructure the three full dictionary files with **63,540 entries** were in `Data/Language Dictionary`.
- Built the Library: language tabs, search, an A to Z list, word details and favorites. The dictionary is large, so the A to Z list is **virtualised** (only about a dozen rows exist and they are re-bound while scrolling), and the whole screen is built in code with a small UI toolkit (`LibraryUiKit`).
- Search ranks exact matches first, then prefix and whole-word matches, then definition matches, so both "house" and "balay" find the same entry.



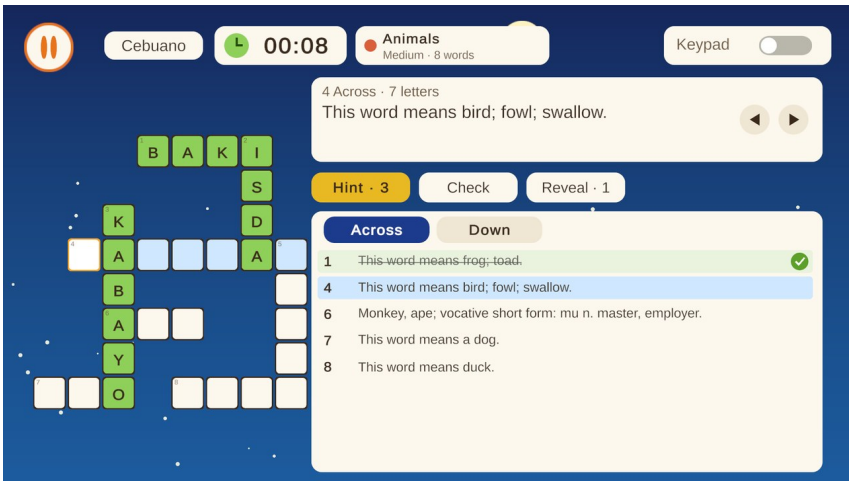
The Library: word of the day, A to Z notebook, favorites

Phase 3. Wordle

- First version: a grid generator (rows and columns set in the Inspector), a keyboard typer, a word verifier that colours tiles, a countdown timer and a dialog panel, with one scene per puzzle.
- Redesign: `WordleTileView` (rounded border and fill, flip and shake animation), `WordleGameController` (clue, hints, on-screen keypad, pause, result card) and `WordleDesignBuilder` to apply the design to the 30 level scenes at once.
- Categories: `WordleCategoryBuilder` created "Wordle - Start" (seven topic cards) and one "Wordle - Play" scene. The level now comes from `WordleSession` and the JSON, set up by `WordleCategoryBootstrap`.

Phase 4. Crossword

- First version: each puzzle was a hand-written C# file (`cebuano - level 1.cs` to `tagalog - level 10.cs`, kept now under Data/Crossword/Legacy levels) read by `CrosswordPuzzleLibrary`.
- Gameplay scripts: `CrosswordGridGenerator` (builds cells from the words), `CrosswordCell`, `CrosswordCluePanel` and `CrosswordClueRow` (Across and Down lists), `CrosswordInputController` (active word, typing, navigation, hints, check, reveal).
- Redesign and categories followed the Wordle pattern (`CrosswordDesignBuilder`, `CrosswordCategoryBuilder`). Puzzles moved to JSON: 7 topics x 3 sizes per language, with grids laid out by a generator (`CrosswordPuzzleGenerator`).



Crossword with the active clue and the Across / Down lists

Phase 5. Word Search

- Built last, so it was data-driven from the start: JSON puzzles, a `WordSearchGenerator` that hides 7 to 8 words in a 10 x 10 grid (across, down, diagonal; seeded so a puzzle always looks the same), a `WordSearchBoardView` that turns a drag into a straight-line selection, and a controller with hints and a result card.
- `WordSearchDesignBuilder` created the Start and Play scenes in the same look as the others.

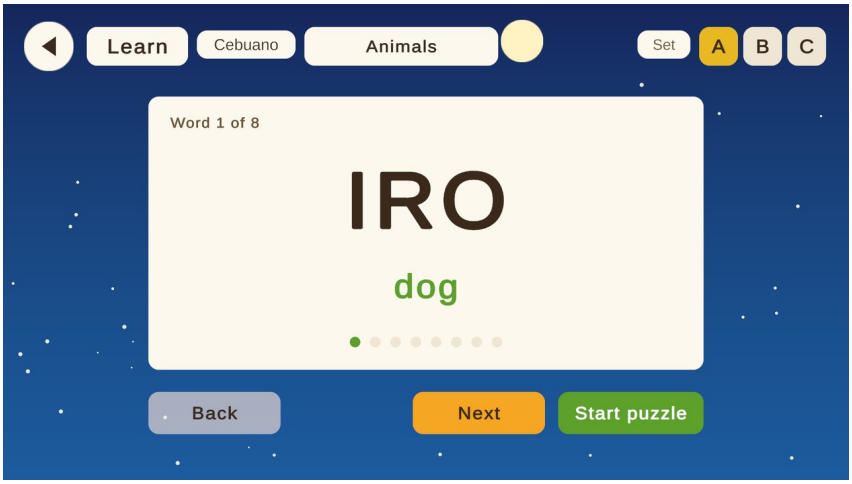
Phase 6. One data model for all games

- All level data moved to JSON, one file per language per game, found through a `LevelDataRegistry` asset in Resources.
- The three games shared one word list per language (133 words), so a meaning always maps to the same word (for example frog is **BAKI** in all three games). Answer lists were printed as review sheets in the `Mockups` folder so every game could be compared with the others by description.
- Local progress stores (PlayerPrefs) recorded solved levels; hub cards show "N of 42 solved" and the Profile shows the same totals.

Phase 7. The learning layer (October 2026)

- The question was "how does a new player learn the words?" The answer was an idea deck and mockups first, then three features.

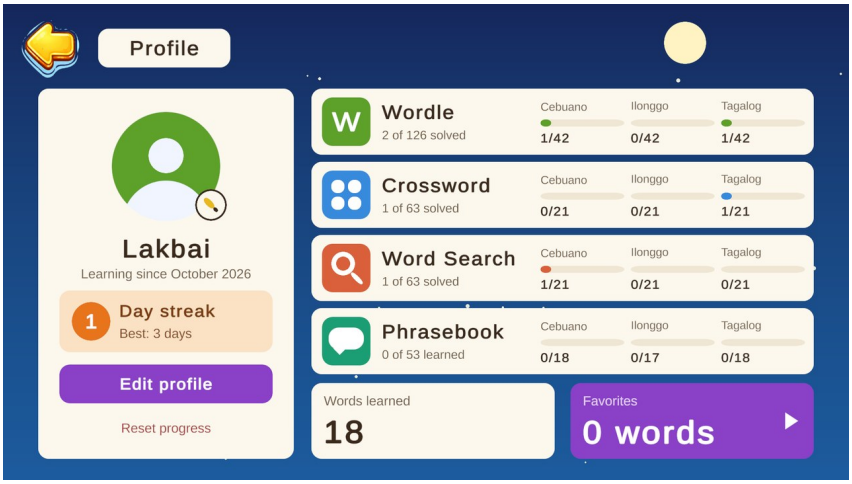
- **Learn cards** (`LearnController`): one card per word with its English meaning, three sets (A, B, C) per topic, and a Start puzzle button.
- **Phrasebook and Sentence builder** (`PhraseTopicsController`, `PhraseCardsController`, `PhraseBuilderController`): seven everyday topics, 53 phrases, tap word tiles into order. First placed on the Profile, then moved to the main menu with its own language chips.
- **Tap to hear** (`WordAudio`): built and installed, then switched off because no recordings exist.



A Learn card: the word first, then the puzzle

Phase 8. Polish and delivery

- **Audio:** seven music tracks and 25 sound effects generated by Python scripts; `BackgroundMusic` and `SoundEffects` create themselves and give every button a sound without per-scene setup.
- **Settings:** a gear button and window (sound effects on or off, music volume slider).
- **Profile redesign:** four compact game rows with a bar per language, words learned, favorites.
- **Screen:** `ScreenFit` limits the window to 800 x 600 to 1024 x 768, letterboxes 4:3 windows, and switches to a 1920 x 1080 full screen window on maximize.
- **Build name:** the Windows build was named Language_Game.exe; Product Name was changed to Lakbay-Salita.



The redesigned Profile (sample data)

5. Technical ideas that hold the project together

Idea	How it works	Why it matters
Session classes	Static classes (WordleSession, CrosswordSession, WordSearchSession, LearnSession, PhraseSession) store the chosen language, topic and level, then load the next scene.	Scenes stay independent; one play scene can play any level.
Level Data Registry	A ScriptableObject in `Resources/LevelDataRegistry.asset` lists the JSON TextAssets of each game in language order.	A language file is swapped by dragging another JSON into a slot.
JSON content	JsonUtility reads `purpose`, `status`, `name` and `categories` from each file.	Writers edit words without touching code.
Progress stores	Small static classes save "language:category:level" lists in PlayerPrefs.	Hub, start screens and Profile agree on what is solved.
Editor builders	Menu items under Tools rebuild scenes from code and are safe to run again.	Design changes apply to every scene at once.
Self-creating singletons	BackgroundMusic, SoundEffects, ScreenFit and WordAudioPlayer start with `RuntimeInitializeOnLoadMethod` and survive scene loads.	Nothing to place in scenes; new scenes work at once.
Procedural UI	`LibraryUiKit` draws rounded cards, circles and stars as sprites in code and offers one-line factory methods.	Most UI needs no image files.
Virtualised list	`LibraryNotebook` keeps about a dozen rows and re-binds them while scrolling.	63,540 words scroll smoothly.
Seeded generation	`WordSearchGenerator` uses a fixed seed per puzzle.	A puzzle looks the same every time.
Time-of-day theme	`MainMenuTheme` blends sky and sea colours by the device clock.	The menu feels alive.
Letterbox	`ScreenFit` sets every camera to the largest 16:9 rectangle and adds a bar camera.	4:3 windows never overlap the layout.

6. Decisions and why

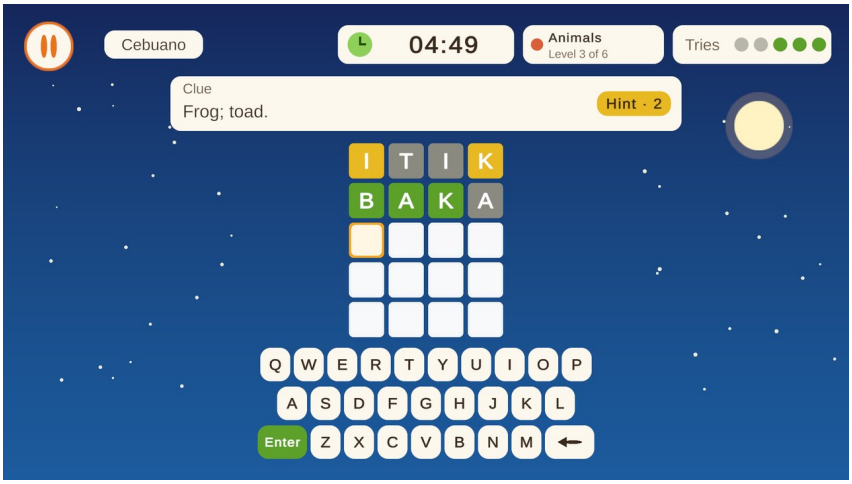
Decision	Reason
Replace per-level scenes and per-level C# data by JSON and one play scene	Changing 60 scenes by hand was slow and error-prone; data is easy to review and extend.
Archive old designs instead of deleting them	Old scenes are kept in `Archive` folders (128 scenes) so a design can be recovered.
Build screens in code with builder tools	Consistent look, repeatable, one place to change a colour.
Learn cards are optional	Experienced players are not slowed down; new players can learn first.
Wordle keypad keys are not coloured	Colouring made it too easy and removed the memory challenge.
Hint wording names the letter and column	"Hint - Letter A is in column 2" is clearer than a bare position.
Phrasebook moved to the main menu with its own language chips	It is a learning area, not a profile detail, and works like the Library tabs.
Phrasebook colour is teal	It clashed with the blue Crossword on the Profile.
Tap to hear switched off	No recordings, and text-to-speech for these languages is unreliable.
Windowed 800 x 600 to 1024 x 768, maximize 1920 x 1080	A fixed, predictable window; the art is 16:9.
One shared word list	One meaning, one word across the games; repetition helps learning.

7. Problems met and how they were solved

Problem	Cause	Fix
The built app was called Language_Game.exe	Product Name in Player Settings was the old project name	Set Product Name to Lakbay-Salita
Same word, different answers (frog = BAKIT in one game, BAKI in another)	Each game had its own word list	Unified the lists and used the shorter form; compared the lists by description
Keypad turned green and gray after each guess	Keyboard colouring was on	Added a `colorKeyboardKeys` switch and turned it off
Hint toast read "Letter 2 is S"	Wording showed a position, not a column	New wording: "Hint - Letter S is in column 2"
Library slider was hard to read	Dark slider on a dark page	Light track and handle
A 4:3 window put the keypad over the grid	Layouts were designed for 16:9	ScreenFit: 16:9 letterbox with a bar camera
A Wordle tile showed UPAT twice	Letters were chosen before the answers were remapped	Compute the used letters after remapping
Profile Phrasebook colour clashed with Crossword	Both were blue	Phrasebook is teal everywhere
Editor code could not delete assets	`AssetDatabase.DeleteAsset` is blocked in the editor-code tool	Removed empty folders on disk, then moved assets
Play-check of Wordle failed from the wrong scene	The session was not set up	Always load the hub first, then the game
Console showed "referenced script (Unknown) is missing"	Investigated: none in active scenes and prefabs	Editor-internal; no change needed
Compiler warning about an unused field in ScreenFit	Fields used only on Windows builds	Wrapped the fields in the same `#if` as the code

8. How the work was tested

- **Play-mode checks:** a helper loaded each scene, clicked buttons, typed letters, found words, and took 1920 x 1080 screenshots. Stages: main menu, Wordle, Crossword, Word Search, Library and Profile.
- **Progress safety:** before a check the saved progress was snapshotted and restored afterwards, so tests did not change real save data.
- **Offline compile:** scripts were also compiled with the Roslyn compiler that ships with Unity, including the Windows-only code (`UNITY_STANDALONE_WIN`).
- **Window sizes:** the editor Game view was set to 1920 x 1080, 1024 x 768 and 800 x 600 to check the layout and letterbox.
- **Data checks:** scripts compared answers between the three games, checked word lengths and counted levels.
- **Not done:** no automated unit tests, and the maximize behaviour has not been tried in a real Windows build.



A screenshot taken by the play-check helper

9. State of the project

9.1 Code by module

Module	Scripts	Lines	What is in it
1 Menu	12	1,951	Main menu motion and theme, settings window, profile, streak chip, hub helpers
2 Wordle	23	4,457	Grid, keyboard, verifier, timer, controller, category scenes, builders
3 Crossword	21	5,207	Grid generator, cells, clue panel, input, puzzle generator, builders
4 Word Search	8	2,316	Generator, board view, controller, start screen, builder
5 Library	7	2,195	Dictionary data and search, notebook list, UI kit
6 Learn	4	774	Learn session, controller, topic button, builder
7 Phrasebook	7	1,666	Data, session, topics, cards, Sentence builder, builder
Audio	5	947	Background music, button sounds, Tap to hear, installer
Core	10	1,059	Player data and database, registry, navigation, streak, ScreenFit

9.2 Known issues

- Words and phrases are **drafts**; they need native-speaker review (Ilonggo phrases especially).
- **Build Settings lists 75 scenes**, 60 of them archived old scenes. Only the 15 playable scenes should be enabled for a release build.
- Maximize and window clamping are untested in a built .exe.
- Game-event sound effects (tile reveal, word found, win, timer) are not connected yet; 18 clips are waiting.
- No recordings for Tap to hear (about 551 needed).
- Some code comments are out of date (for example the registry comment still names old script folders, and the Wordle controller comment still mentions coloured keys).
- Legacy folders remain: `Assets/Sub`, the per-level legacy data and the archived scenes.

10. Lessons and next steps

10.1 What worked

- Moving content into JSON early made the late features (Phrasebook, Word Search, Learn) cheap to add.
- Builder scripts made big visual changes safe; archived scenes made them reversible.
- Showing an idea as an image before building avoided rework.
- Self-creating singletons removed per-scene wiring for audio and window handling.

10.2 What to do differently

- Move to the data-driven model from the first game instead of the third.
- Keep Build Settings clean from the start and archive scenes outside it.
- Add edit-mode tests for the data classes (counts, word lengths, unique answers).
- Write a short changelog with each commit; early commit messages carry no information.

10.3 Next steps

Order	Task
1	Native-speaker review of every word, clue and phrase
2	Test a Windows build (maximize, window limits, saving)
3	Clean Build Settings to the 15 playable scenes
4	Record words and switch on Tap to hear; connect game-event sound effects
5	Commit the pending work to sub (and then main) and playtest with real learners

Appendix. Editor tools (Tools menu)

Menu item	What it does
Tools > Wordle > Build category scenes and hub tag	Creates Wordle - Start and Wordle - Play, wires the hub tag
Tools > Wordle > Rebuild design in open scene / all 30 level scenes	Reapplies the Wordle design
Tools > Crossword > Archive current crossword and build category scenes	Archives the 2nd design, creates Crossword - Start and Play
Tools > Crossword > Rebuild design in open scene / all 30 level scenes	Reapplies the Crossword design
Tools > Word Search > Build Start and Play scenes	Creates the two Word Search scenes
Tools > Learn > Build Learn scene and buttons	Creates Learn - Cards and the Learn buttons on topic cards
Tools > Phrasebook > Build Phrasebook scenes and menu card	Creates the three Phrasebook scenes and the main menu card
Tools > Settings > Build gear button and settings window	Replaces the music chip by the gear and builds the settings window
Tools > Profile > Rebuild design in open scene / Archive current design and rebuild	Rebuilds the Profile screen
Tools > Audio > Add / Show / Hide Tap to hear buttons	Installs or toggles the speaker buttons